

Question description

You have an undirected graph consisting of n vertices with weighted edges.

A simple cycle is a cycle of the graph without repeated vertices. Let the *weight* of the cycle be the **XOR** of weights of edges it consists of.

Let's say the graph is *good* if all its *simple* cycles have weight 1. A graph is bad if it's not good.

Initially, the graph is empty. Then q queries follow. Each query has the next type:

- $u \ v \ x$ — add edge between vertices u and v of weight x if it doesn't make the graph bad.

For each query print, was the edge added or not.

Constraints

$$3 \leq n \leq 3 \cdot 10^5; 1 \leq q \leq 5 \cdot 10^5$$

$$1 \leq u, v \leq n; u \neq v; 0 \leq x \leq 1$$

Input

The first line contains two integers n and q — the number of vertices and queries.

Next q lines contain queries — one per line. Each query contains three integers u , v and x — the vertices of the edge and its weight.

It's guaranteed that there are no multiple edges in the input.

Output

For each query, print YES if the edge was added to the graph, or NO otherwise (both case-insensitive).

Logical Test Cases

Test Case 1

INPUT (STDIN)

9 12
6 1 0
1 3 1
3 6 0
6 2 0
6 4 1
3 4 1
2 4 0
2 5 0
4 5 0
7 8 1
8 9 1
9 7 0

EXPECTED OUTPUT

YES
YES
YES
YES
YES
NO
YES
YES
NO
YES
YES
NO

Test Case 2

INPUT (STDIN)

9 6
1 3 1
3 6 0
6 4 1
2 4 0
4 5 0
7 8 1

EXPECTED OUTPUT

YES
YES
YES
YES
YES
YES

Mandatory Test Cases

Test Case 1

KEYWORD

int dfs1(int np,int lst)

✓ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

12

Test Case 2

TOKEN COUNT

1255

Test Case 3

NLOC

78

Graph5.c +

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define MAXN 300005
6
7 typedef struct Edge {
8     int to;
9     int weight;
10    int id;
11    int in_cycle;
12    struct Edge *nxt;
13    struct Edge *rev;
14} Edge;
15
16 Edge *head[MAXN];
17 int visited[MAXN];
18 int path_nodes[MAXN];
19 Edge *path_edges[MAXN];
20 int path_len = 0;
21 int target_node = 0;
22 int found = 0;
23
24 void add_edge_graph(int u, int v, int w, int id) {
25     Edge *e1 = (Edge*)malloc(sizeof(Edge));
```

```
26 Edge *e2 = (Edge*)malloc(sizeof(Edge));
27
28 e1->to = v;
29 e1->weight = w;
30 e1->id = id;
31 e1->in_cycle = 0;
32 e1->nxt = head[u];
33 e1->rev = e2;
34 head[u] = e1;
35
36 e2->to = u;
37 e2->weight = w;
38 e2->id = id;
39 e2->in_cycle = 0;
40 e2->nxt = head[v];
41 e2->rev = e1;
42 head[v] = e2;
43}
44
45int dfsi(int np, int lst) {
```

```

46     visited[np] = 1;
47     if (np == target_node) {
48         found = 1;
49         return 1;
50     }
51     for (Edge *e = head[np]; e; e = e->nxt) {
52         int v = e->to;
53         if (v != lst && !visited[v]) {
54             path_edges[path_len] = e;
55             path_nodes[path_len] = v;
56             path_len++;
57             if (dfs1(v, np)) return 1;
58             path_len--;
59         }
60     }
61     return 0;
62 }
63
64 int parent_dsu[MAXN];
65 int find_dsu(int i) {

```

```
66     if (parent_dsu[i] == i) return i;
67     return parent_dsu[i] = find_dsu(parent_dsu[i]);
68 }
69
70 int main() {
71     int n, q;
72     if (scanf("%d %d", &n, &q) != 2) return 0;
```

main | CH.SC.U4CSE25225 | Graph5.c

Ln 143, Col 1 Spaces: 4 UTF-8 CRLF C

```
73
74     for (int i = 1; i <= n; i++) {
75         parent_dsu[i] = i;
76         head[i] = NULL;
77     }
78
79     int edge_id_counter = 0;
80
81     for (int k = 0; k < q; k++) {
82         int u, v, x;
83         if (scanf("%d %d %d", &u, &v, &x) != 3) break;
84
85         int ru = find_dsu(u);
```

```
86     int rv = find_dsu(v);
87
88     if (ru != rv) {
89         parent_dsu[ru] = rv;
90         add_edge_graph(u, v, x, ++edge_id_counter);
91         printf("YES\n");
92     } else {
93         path_len = 0;
94         target_node = v;
95         found = 0;
96         for (int i = 1; i <= n; i++) visited[i] = 0;
97
98         path_nodes[0] = u;
99         dfsi(u, 0);
100
101         int has_cycle_edge = 0;
102         int xor_sum = 0;
103         for (int i = 0; i < path_len; i++) {
104             if (path_edges[i]->in_cycle) {
105                 has_cycle_edge = 1;
```

```

106         break;
107     }
108     xor_sum ^= path_edges[i]->weight;
109 }
110
111 if (!has_cycle_edge && (xor_sum ^ x) == 1) {
112     for (int i = 0; i < path_len; i++) {
113         path_edges[i]->in_cycle = 1;
114         path_edges[i]->rev->in_cycle = 1;
115     }
116     Edge *new_e1 = (Edge*)malloc(sizeof(Edge));
117     Edge *new_e2 = (Edge*)malloc(sizeof(Edge));
118     new_e1->to = v;
119     new_e1->weight = x;
120     new_e1->id = ++edge_id_counter;
121     new_e1->in_cycle = 1;
122     new_e1->nxt = head[u];
123     new_e1->rev = new_e2;
124     head[u] = new_e1;

```

```
125
126         new_e2->to = u;
127         new_e2->weight = x;
128         new_e2->id = edge_id_counter;
129         new_e2->in_cycle = 1;
130         new_e2->nxt = head[v];
131         new_e2->rev = new_e1;
132         head[v] = new_e2;
133
134         printf("YES\n");
135     } else {
136         printf("NO\n");
137     }
138 }
139 }
140
141 return 0;
142 }
143
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt
```

```
9 12
```

```
6 1 0
```

```
1 3 1
```

```
3 6 0
```

```
6 2 0
```

```
6 4 1
```

```
3 4 1
```

```
2 4 0
```

```
2 5 0
```

```
4 5 0
```

```
7 8 1
```

```
8 9 1
```

```
9 7 0
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph5.exe
```

```
YES
```

```
YES
```

```
YES
```

```
YES
```

```
YES
```

```
NO
```

```
YES
```

```
YES
```

```
NO
```

```
YES
```

```
YES
```

```
NO
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>
```

```
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt
```

```
9 6
1 3 1
3 6 0
6 4 1
2 4 0
4 5 0
7 8 1
C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Graph5.exe
```

YES

YES

YES

YES

YES

YES

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>